BÖLÜM 8 / MODULE 8

Klasik MAANG Tasarım Problemleri - Bölüm 2

Classic MAANG System Designs - Part 2

WhatsApp/Slack Sohbet Sistemi, Twitter/Instagram News Feed (Fan-Out), YouTube Video Akışı (HLS/DASH), Dropbox/Drive Senkronizasyonu, Google Typeahead (Trie) ve Flaş İndirim (Ticketmaster).

WhatsApp/Slack Chat (WebSockets), Twitter News Feed (Fan-Out trade-offs), YouTube Video Streaming (HLS/DASH), Dropbox File Sync (Block Chunks), Google Search Typeahead (Trie), and Ticketmaster Flash Sales.

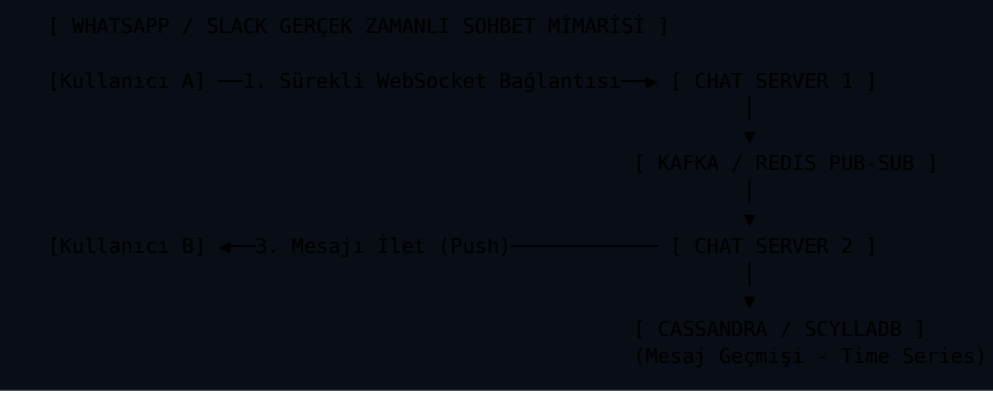
43

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

Gerçek Zamanlı Sohbet Sistemi (WhatsApp / Slack): WebSockets, Presence ve Cassandra

Designing a Chat System (WhatsApp / Slack): WebSockets & Presence

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski internet sitelerinde bir arkadaşınızın size mesaj yazıp yazmadığını görmek için sayfayı her 3 saniyede bir 'Yenile' (Polling) yapardınız (kapıyı 100 kez açıp kapatmak gibi). WhatsApp ve Slack ise iki ev arasına çekilmiş kesintisiz bir telefon hattı gibidir (WebSocket). Hat bir kez açılır ve 24 saat boyunca açık kalır. Arkadaşınız 'Selam' dediği anda ses kulağınızda yankılanır. 'Çevrimiçi / Online' göstergesi ise kapının önündeki lambadır: Siz odaya girdiğinizde lamba yanar (Presence Server); 30 saniye boyunca hareket etmezseniz lamba otomatik söner!

🇬🇧 ENGLISH (ELI5)

In early web chat rooms, checking for new messages meant mashing the 'Refresh' button every 3 seconds (HTTP Polling)—like constantly slamming your front door open and shut. Modern chat apps like WhatsApp and Slack deploy a persistent, open fiber-optic intercom (WebSocket connection). The duplex socket opens once and stays active. When a friend types 'Hi', the byte stream transmits instantly. The 'Online' status indicator is a motion sensor: when your phone pings the presence gateway, the light glows green; if no heartbeat arrives for 30 seconds, it shifts to offline!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Sohbet Mimarisi Bileşenleri: 1) İletişim Protokolü: İki yönlü, düşük gecikmeli, başlık yükü minimum WebSocket protokolü kullanılır. 2) Durum Takibi (Presence Service): Kullanıcının çevrimiçi/çevrimdışı durumunu yönetir. İstemciler her 10-15 saniyede bir hafif Heartbeat (Canlılık sinyali) gönderir. Heartbeat kesilirse Redis'teki oturum süresi (TTL) dolar ve kullanıcı 'Offline' işaretlenir. 3) Mesaj Depolama (Neden Cassandra?): Sohbet mesajları zaman serisi (Time-series) verisidir; yazma hacmi aşırı yüksektir, eski mesajlar nadiren güncellenir. Cassandra'nın LSM-Tree mimarisi saniyede milyonlarca mesaj yazmayı mükemmel kaldırır. Bölümleme: `Partition Key: chat_id, Clustering Key: message_id (Snowflake)` ile mesajlar kronolojik sırada diske dökülür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Chat System Topology: 1) Transport: Full-duplex stateful WebSockets over TCP, avoiding HTTP connection handshake penalties. 2) Presence Cluster: Tracks ephemeral user heartbeats. Clients dispatch heartbeat pings every 15s. Absence of pings expires a Redis lease, broadcasting offline status to user contact channels. 3) Storage Engine (Why Cassandra / ScyllaDB?): 1-on-1 chat history exhibits append-only sequential access patterns with billions of writes per day. Relational databases choke on index rebalancing. Cassandra maps `Partition Key = conversation_id` with `Clustering Key = message_id (Snowflake time-ordered)`, storing millions of chronological messages sequentially on disk for lightning-fast pagination.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: WebSocket Oturum Eşleme ve Chat Yönlendirici Modeli
import java.util.concurrent.*;

public class ChatSessionRouter {
    // Aktif WebSocket sunucuları haritası: UserID -> Hangi Chat Server'a bağlı?
    private final ConcurrentMap<String, String> userServerRegistry = new ConcurrentHashMap<>();

    public void registerSession(String userId, String serverId) {
        userServerRegistry.put(userId, serverId);
        IO.println("Kullanıcı " + userId + " bağlandı -> Sunucu: " + serverId);
    }

    public String routeMessage(String recipientId) {
        String server = userServerRegistry.get(recipientId);
        if (server != null) {
            return "FORWARD_WEBSOCKET -> " + server;
        }
        return "DISPATCH_APNS_OR_FCM_PUSH (Kullanıcı Çevrimdışı!)";
    }
}

void main() {
    var router = new ChatSessionRouter();
    router.registerSession("user_ali", "chat-gateway-pod-1");

    IO.println("Mesaj Yönlendirme (Ali):  " + router.routeMessage("user_ali"));
    IO.println("Mesaj Yönlendirme (Ayşe): " + router.routeMessage("user_ayse"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 100.000 kişilik bir grupta biri mesaj attığında 'Mesajı gruptaki 100.000 kişinin WebSocket soketine tek tek yazarım' demeyin! Bu 'Fan-out' darbesi sunucuyu kilitler. Büyük gruplarda mesajlar merkezi bir kuyruğa yazılır ve istemciler kendi pencereleri açıldığında mesajları çeker.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Pushing group messages immediately to all 100,000 members of a mega-channel concurrently over WebSockets introduces a fatal fan-out write storm that saturates network buffers. Large channels require pull-based pagination or aggregated pub/sub partitions.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- WebSocket sürekli açık kalan iki yönlü TCP kanalıyla anlık sohbetin omurgasıdır.
- Kullanıcı durumu (Presence) periyodik Heartbeat ve Redis TTL anahtarlarıyla yönetilir.
- Sohbet geçmişi için Cassandra/ScyllaDB LSM-Tree mimarisiyle en yüksek yazma hızını sunar.

🎯 Key Takeaways for the Interview

- Persistent WebSockets eliminate handshake overhead for sub-50ms message delivery.
- Ephemeral user presence states are managed via heartbeat loops and auto-expiring Redis keys.
- Cassandra's sequential append-only LSM-Tree storage is the gold standard for chat persistence.

44

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

Sosyal Medya Haber Akışı (News Feed / Twitter): Fan-Out-on-Write vs Fan-Out-on-Read

Designing a News Feed (Twitter / Instagram): Fan-Out Architectures

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ 1. FAN-OUT-ON-WRITE (Push Modeli - Normal Kullanıcılar İçin) ]
Ali Tweet Atar ──► Takipçileri Bul (150 kişi) ──► Her birinin Feed Önbelleğine Yaz!
(Okuma anında akış hazır bekler: O(1) okuma hızı!)

[ 2. FAN-OUT-ON-READ (Pull Modeli - Ünlüler / Celebrities İçin) ]
Cristiano Ronaldo Tweet Atar (600 Milyon Takipçi!)
600 Milyon önbelleğe yazmaya kalkarsanız sunucular çöker (Celebrity Problem)!
Çözüm: Ronaldo'nun tweet'i sadece kendi profiline yazılır. Takipçi akışını açtığında
normal akış ile Ronaldo'nun tweet'i dinamik olarak harmanlanır (Merge)!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Haber akışı oluşturmanın iki yolu vardır: 1) Fan-Out-on-Write (Gazete Dağıtıcısı Modeli): Mahallenin muhtarı bir duyuru yaptığında, çırak gidip mahalledeki 50 evin kapısının altından duyuruyu atar. Sabah uyanan herkes kapının önündeki duyuruyu saniyesinde okur (Okuma anında sıfır bekleme!). 2) Fan-Out-on-Read (Ünlü Problemi): Eğer Cristiano Ronaldo veya Taylor Swift duyuru yaparsa ne olur? 600 milyon takipçinin evine tek tek kağıt dağıtmaya kalkarsanız posta teşkilatı iflas eder! Ünlüler duyuruyu meydana asar; siz sabah sokağa çıktığınızda kendi mektuplarınızla meydandaki büyük afişi birleştirip okursunuz!

🇬🇧 ENGLISH (ELI5)

Two contrasting architectures for social feeds: 1) Fan-Out-on-Write (Push Model): When an ordinary user with 200 friends posts a photo, background workers push a reference into each of those 200 friend's private in-memory feed caches. When a friend opens their phone, their timeline is already pre-computed (O(1) read latency!). 2) Fan-Out-on-Read (Pull Model / The Celebrity Problem): When Cristiano Ronaldo or Taylor Swift posts to 600 million followers, pushing to 600M individual caches burns terabytes of RAM and locks queues for hours! Instead, store the celebrity tweet once on their wall; when a fan opens their feed, the app dynamically merges their friends' cached feed with Ronaldo's latest tweet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG News Feed Mimarisi (Hibrit Model): 1) Fan-Out-on-Write (Push): Normal kullanıcılar (<25.000 takipçi) için geçerlidir. Kullanıcı tweet attığında bir worker takipçilerin Redis ZSET akışlarına tweet ID'sini ekler. Okuma $O(1)$'dir. 2) Fan-Out-on-Read (Pull): Ünlüler (Celebrity / VIP kullanıcılar) için geçerlidir. Tweet takipçilerin önbelleğine yazılmaz; sadece ünlünün kendi kütüğüne yazılır. İstemci akışını yenilediğinde, önbelleğindeki akış ile takip ettiği ünlülerin en son tweetleri birleştirilerek (K-Way Merge) kullanıcıya sunulur. Bu hibrit mimari hem Twitter'ın hem de Instagram'ın omurgasıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Hybrid News Feed Architecture: 1) Fan-Out-on-Write (Push): Applied to standard users (<25,000 followers). Asynchronous workers fan out the post ID into follower timeline caches (Redis ZSET keyed by timestamp). Delivers $O(1)$ instantaneous feed generation. 2) Fan-Out-on-Read (Pull): Applied to high-degree celebrities (e.g. 50M+ followers). Writing to 50M Redis sets exhausts cluster bandwidth. The celebrity tweet is stored only in their user timeline; when an active follower requests their feed, the system executes an in-memory K-way merge between the user's pre-computed inbox and their followed celebrities' latest posts. This hybrid approach represents the production standard at Twitter and Instagram.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Hibrit Fan-Out Karar Motoru (Celebrity Problem Çözümü)
public class HybridFeedFanOut {
    private static final int CELEBRITY_THRESHOLD = 50_000;

    public record User(String id, String username, int followerCount) {
        public boolean isCelebrity() { return followerCount >= CELEBRITY_THRESHOLD; }
    }

    public String determineFanOutStrategy(User author) {
        if (author.isCelebrity()) {
            return "PULL_STRATEGY: Takipçilere yazma! Sadece ünlünün profiline kaydet. (Followers: " +
author.followerCount() + ")";
        } else {
            return "PUSH_STRATEGY: " + author.followerCount() + " takipçinin Redis Feed önbelleğine
asenkron fırlat!";
        }
    }
}

void main() {
    var engine = new HybridFeedFanOut();
    var normalUser = new HybridFeedFanOut.User("u1", "ali_bozkurt", 240);
    var celebrity = new HybridFeedFanOut.User("u2", "elon_musk", 180_000_000);

    IO.println("Ali için Strateji:  " + engine.determineFanOutStrategy(normalUser));
    IO.println("Elon için Strateji: " + engine.determineFanOutStrategy(celebrity));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Her tweet atıldığında takipçilerin feed'ini yeniden hesaplarım (Fan-out on Read)' derseniz sistem 100M kullanıcıda çöker; 'Herkes için Fan-out on Write yaparım' derseniz Elon Musk tweet attığında sistem çöker! Çözüm: Kesinlikle HİBRİT modeldir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Choosing purely Fan-out on Read causes database exhaustion during timeline loads; choosing purely Fan-out on Write destroys queues when celebrities post. Propose the HYBRID model: push for ordinary users, pull for celebrities.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Fan-out on Write okuma hızını O(1) seviyesine indirir ancak ünlülerde patlar.
- Celebrity Problem, takipçi sayısı yüksek hesapların akışını okuma anında dinamik harmanlayarak çözülür.
- Haber akışları Redis Sorted Sets (ZSET) veri yapısında zaman damgası skoruyla tutulur.

🎯 Key Takeaways for the Interview

- Fan-out on Write provides instantaneous O(1) timeline reads but stumbles on celebrity fan-out.
- The Celebrity Problem is resolved by dynamically blending hot celebrity timelines on read.
- User feed inboxes are maintained inside Redis Sorted Sets keyed by epoch millisecond scores.

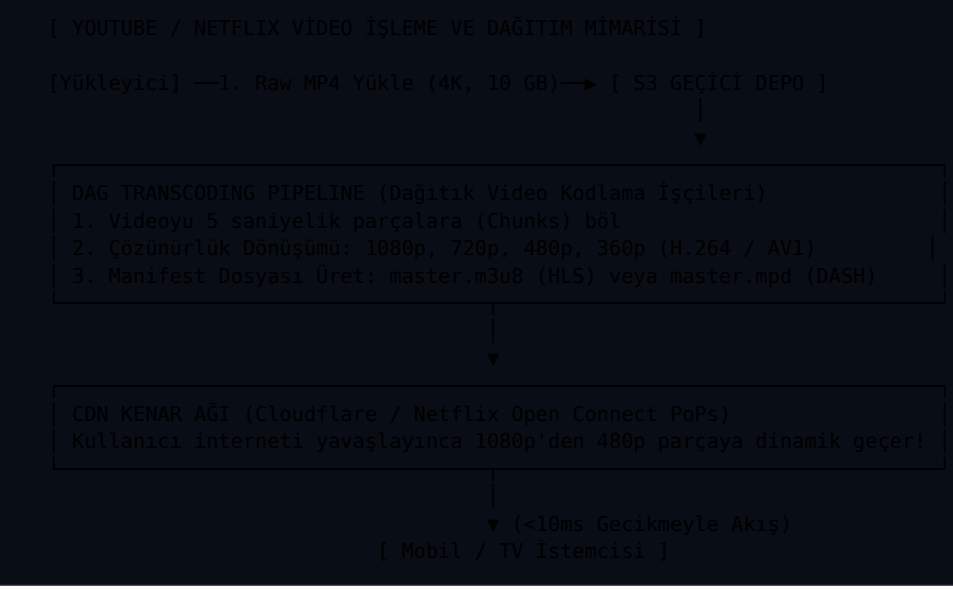
45

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

Video Akış Platformu (YouTube / Netflix) Tasarımı: Transcoding, Chunking ve HLS/DASH

Designing Video Streaming (YouTube / Netflix): HLS/DASH & Transcoding

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

YouTube'a 10 GB'lık dev bir film yüklediğinizde, bir kullanıcının o filmi tek parça halinde indirmesini beklemezsiniz; telefonun hafızası dolar ve interneti donardı! YouTube videoyu bir ekmek gibi 5 saniyelik dilimlere (Chunks) böler. Üstelik her dilimin 4K, 1080p, 720p ve 360p olmak üzere 4 farklı kalitede kopyasını çıkarır (Transcoding). Siz metroya binip internetiniz zayıfladığında video durmaz; oynatıcı anında 1080p dilim almayı bırakıp 360p dilimi almaya başlar (Adaptif Akış - HLS)! Metrodan çıkınca tekrar kristal netliğinde 4K dilimlere geri döner!

🇬🇧 ENGLISH (ELI5)

When uploading a 10GB 4K raw film to YouTube, you don't stream that gargantuan file in a single monolithic blob; mobile devices would freeze and crash! YouTube slices the video like a loaf of bread into hundreds of 5-second crumbs called 'Chunks'. Furthermore, an automated transcoding farm converts each crumb into 1080p, 720p, 480p, and 360p bitrates. When you step into an elevator and mobile signal weakens, playback doesn't halt; the player dynamically switches from fetching 1080p chunks to 360p chunks (Adaptive Bitrate Streaming via HLS/DASH) seamlessly!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Video Platformu Mimarisi: 1) Video Yükleme ve DAG Pipeline: Video S3'e yüklendikten sonra bir Kafka olayı dağıtık kodlama motorunu (Transcoding Cluster - FFmpeg) tetikler. DAG (Directed Acyclic Graph) yapısıyla video ses, altyazı ve video parçalarına ayrıştırılır. 2) Adaptif Bit Hızı Akışı (Adaptive Bitrate Streaming - ABR): Endüstri standartları Apple HLS (`.m3u8` manifest ve `.ts` segmentleri) ve MPEG-DASH'tir (`.mpd`). Oynatıcı istemci, ağ bant genişliğini milisaniye seviyesinde ölçer ve bir sonraki 5 saniyelik parçayı en uygun çözünürlükten ister. 3) CDN Dağıtımı: Video segmentleri değiştirilemez (Immutable) statik dosyalardır; CDN PoP noktalarında %99 oranında önbelleğe alınır. Netflix'in Open Connect mimarisi, popüler filmleri gece saatlerinde doğrudan yerel telekom ISP'lerinin veri merkezlerindeki sunuculara kopyalar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architecture of Scale Video Platforms (YouTube / Netflix): 1) Video Ingestion & DAG Transcoding: Uploaded raw video drops into an ingestion S3 bucket, spawning a Directed Acyclic Graph (DAG) workflow across GPU compute worker pools. Tasks parallelize video chunking, audio extraction, watermarking, and multi-codec encoding (H.264, H.265, AV1). 2) Adaptive Bitrate Streaming (ABR): Packaged via HLS (`.m3u8` playlists and `.ts` chunk files) or MPEG-DASH (`.mpd`). Client players dynamically monitor throughput and buffer depths, swapping bitrates up or down mid-stream without frame drops. 3) Edge CDN Distribution: Chunks are immutable static files cached at edge PoPs with 98%+ cache hit rates. Netflix Open Connect appliances deploy custom storage servers directly inside thousands of residential ISP data centers globally.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: HLS Manifest ve Adaptif Segment Çözümleyici Modeli
public class HlsManifestEngine {
    public record VideoSegment(int sequenceNumber, String resolution, String chunkUrl, int
durationSecs) {}

    public static VideoSegment selectAdaptiveChunk(int currentBandwidthKbps, int chunkIndex) {
        // İstemcinin anlık internet hızına göre en iyi çözünürlüğü seç:
        String quality = switch (currentBandwidthKbps) {
            case int b when b >= 8000 -> "4K_2160p";
            case int b when b >= 3000 -> "FHD_1080p";
            case int b when b >= 1200 -> "HD_720p";
            default                   -> "SD_360p";
        };
        return new VideoSegment(chunkIndex, quality, "/video/chunk_" + chunkIndex + "_" + quality +
".ts", 5);
    }
}

void main() {
    // Metroya girildi: Hız 9000 Kbps'den 800 Kbps'e düştü!
    var highSpeedChunk = HlsManifestEngine.selectAdaptiveChunk(9500, 42);
    var lowSpeedChunk = HlsManifestEngine.selectAdaptiveChunk(800, 43);

    IO.println("Evdeki Parça (Wi-Fi):  " + highSpeedChunk.resolution() + " -> " +
highSpeedChunk.chunkUrl());
    IO.println("Metrodaki Parça (Edge):" + lowSpeedChunk.resolution() + " -> " +
lowSpeedChunk.chunkUrl());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Videoları tek bir sunucuda FFmpeg ile dönüştürürüm' demeyin! 2 saatlik bir 4K video tek makinede saatlerce sürer. Çözüm: Video 5 dakikalık parçalara bölünür, 50 ayrı makinede paralel kodlanır ve sonunda manifest dosyasında birleştirilir (MapReduce tarzı paralel video işleme).

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Proposing a single server running FFmpeg to transcode 4K videos is a fatal architectural flaw. Slice raw video into discrete chunks, fan out encoding across distributed GPU worker fleets in parallel, and stitch the master manifest together.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- HLS ve DASH adaptif protokolleri videoyu 5 saniyelik parçalar halinde sunar.
- İstemci oynatıcı ağ hızına göre sonraki parçanın çözünürlüğünü dinamik olarak seçer.
- Video parçaları değiştirilemez statik dosyalar olduğu için CDN'lerde %98 oranında önbelleğe alınır.

🎯 Key Takeaways for the Interview

- HLS and DASH slice video into 5-second chunks for adaptive bitrate switching.
- Client players probe network throughput continuously to fetch optimal chunk resolutions.
- Immutable video segments enable near-perfect CDN edge cache hit ratios.

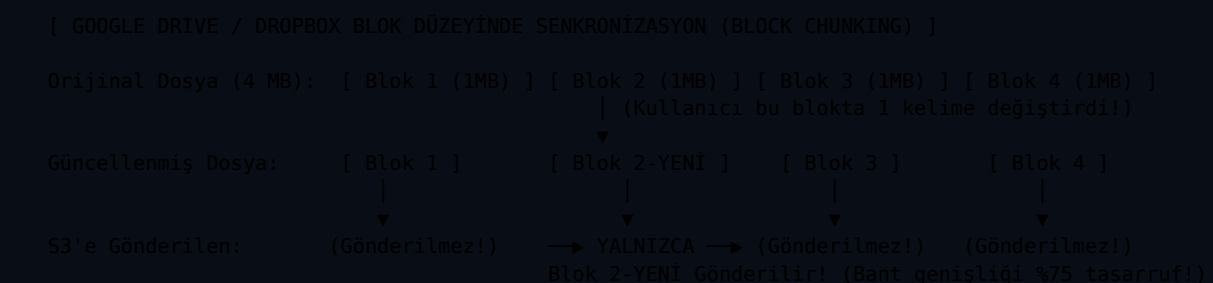
46

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

Dosya Depolama ve Senkronizasyon (Google Drive / Dropbox): Chunking ve Sync

Designing Cloud Storage (Google Drive / Dropbox): Chunking & Sync

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

100 sayfalık kalın bir kitap üzerinde çalıştığınızı düşünün. 52. sayfada tek bir harf hatasını düzelttiğinizde koca kitabı matbaaya gönderip 100 sayfayı baştan bastırmazsınız; sadece 52. sayfayı değiştirirsiniz! Dropbox ve Google Drive da böyle çalışır: 2 GB'lık bir video dosyasını 4 MB'lık yüzlerce bloğa (Chunk) böler. Dosyada tek bir satır değiştiğinde 2 GB'ı baştan yüklemez; sadece değişen o 4 MB'lık küçük bloğu bulur ve buluta fırlatır (Delta Sync). İnternet kotanız ve zamanınız kurtulur!

🇬🇧 ENGLISH (ELI5)

Editing a single typo on page 52 of a 500-page book doesn't require incinerating the entire print run and re-printing all 500 pages; you replace page 52! Cloud storage providers like Dropbox and Google Drive partition large files into discrete 4MB chunks. When you modify one sentence in a 1GB file, the desktop sync engine calculates cryptographic hashes across all chunks, identifies the single mutated block, and uploads only that 4MB delta! Bandwidth consumption drops by 99%!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Dosya Depolama Mimarisi: 1) Blok Düzeyinde Bölme (Chunking): Dosyalar 4 MB'lık bloklara bölünür. Her bloğun SHA-256 hash'i alınır. 2) Küresel Tekilleştirme (Cross-User Deduplication): Eğer Ahmet ve Mehmet aynı 1 GB'lık Ubuntu ISO dosyasını yüklüyorsa, sistem blok hash'lerine bakar. Bloklar zaten S3'te varsa İKİNCİ KEZ YÜKLENMEZ; sadece veritabanında referans pointer'ı açılır. Milyonlarca dolar depolama ve ağ tasarrufu sağlanır. 3) Senkronizasyon Çakışması (Sync Conflict): İki kullanıcı çevrimdışı aynı dosyayı düzenleyip aynı anda bağlanırsa ne olur? Git tarzı versiyon çakışması üretilir: 'Dosya (Ali's conflicted copy)' şeklinde yeni bir dosya dallandırılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Cloud Storage & Sync Architecture: 1) Block-Level Chunking: Files split into fixed (4MB) or variable-length (Rabin fingerprinting) chunks. Each block generates a unique cryptographic SHA-256 hash. 2) Cross-User Deduplication: If 10,000 corporate employees save the same 500MB PDF presentation, block hashing detects matching fingerprints in S3; the storage cluster skips the upload entirely and increments metadata reference pointers, saving petabytes of storage. 3) Delta Sync Engine: Local sync daemons monitor file system watch events, compute modified block diffs, and upload strictly modified chunks. 4) Conflict Resolution: Concurrent offline mutations are resolved by branching diverging file trees ('Project Plan (Alice's conflicted copy)').

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Blok Düzeyinde SHA-256 Hash Hesaplama ve Tekilleştirme Simülasyonu
import java.security.MessageDigest;
import java.util.*;

public class BlockDeduplicationEngine {
    private final Set<String> cloudBlockStorage = new HashSet<>();

    public String computeBlockHash(byte[] blockBytes) throws Exception {
        var md = MessageDigest.getInstance("SHA-256");
        byte[] digest = md.digest(blockBytes);
        return HexFormat.of().formatHex(digest).substring(0, 12); // Kısaltılmış hash
    }

    public boolean uploadBlockIfMissing(String blockHash) {
        // Blok zaten S3'te varsa YÜKLEME! (Deduplication)
        return cloudBlockStorage.add(blockHash);
    }
}

void main() throws Exception {
    var engine = new BlockDeduplicationEngine();
    byte[] chunk1 = "Ders Notu Parça 1".getBytes();
    String hash1 = engine.computeBlockHash(chunk1);

    IO.println("Blok 1 Yüklendi mi (İlk Kez)? " + engine.uploadBlockIfMissing(hash1));
    IO.println("Blok 1 Yüklendi mi (Tekrar)?  " + engine.uploadBlockIfMissing(hash1) + "
(Deduplication Devrede!)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Dosyaları tek parça halinde yüklerim' demeyin! 5 GB'lık bir dosya yüklenirken %99'da internet koparsa tüm dosya çöpe gider. Blok tabanlı yüklemede (Chunking) sadece kopan 4 MB'lık blok yeniden denenir (Resumable Upload).

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch uploading monolithic multi-gigabyte files in single HTTP streams. If connectivity drops at 99%, the entire upload fails. Chunking guarantees resumable uploads where only the interrupted 4MB block re-transmits.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Dosyaları 4MB'lık bloklara bölmek yalnızca değişen kısımların yüklenmesini (Delta Sync) sağlar.
- Blok hash'leri sayesinde aynı dosya bulutta tek bir kez saklanır (Cross-User Deduplication).
- Kopan yüklemeler baştan başlamaz; yalnızca eksik kalan bloklar kaldığı yerden devam eder.

🎯 Key Takeaways for the Interview

- Chunking files into 4MB blocks enables delta sync, uploading only modified bytes.
- Cross-user deduplication detects identical block hashes, eliminating petabytes of duplicate S3 data.
- Chunked uploads enable robust resumable uploads after network disconnects.

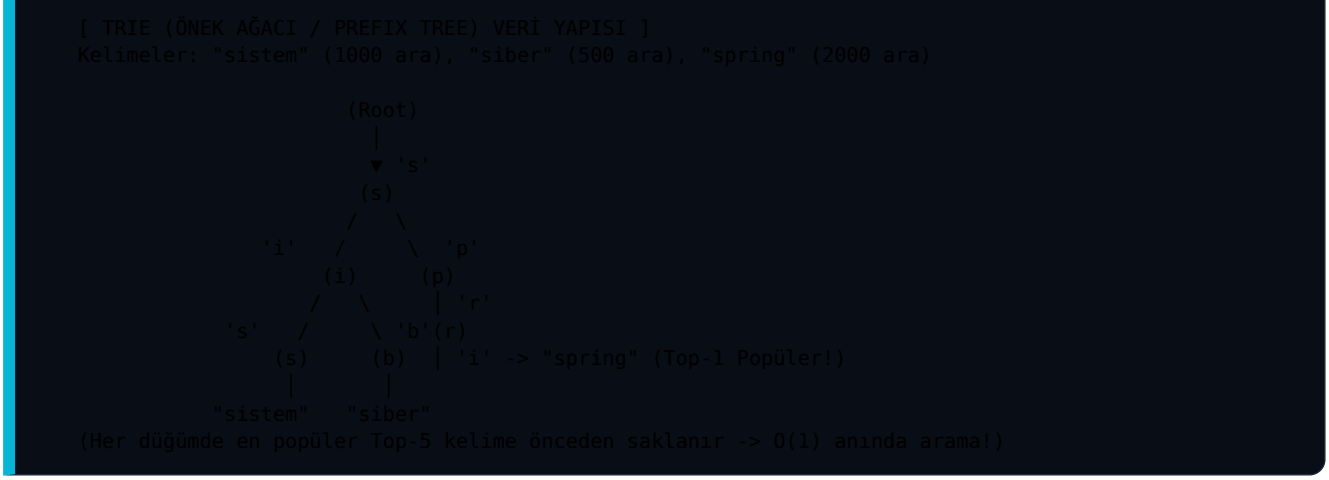
47

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

Arama Otomatik Tamamlama (Typeahead / Google Suggestion): Trie ve Redis ZSET

Designing Search Typeahead (Google Suggestion): Trie & Redis ZSET

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Google'ın arama kutusuna 'sis' yazdığınız anda kutunun altında 'sistem tasarımı', 'siber güvenlik' gibi önerilerin şak diye belirmesini düşünün. Dünyada 1 milyar kelime varken Google her harf bastığınızda tüm veritabanını tarayamaz! Bunun yerine akıllı bir soy ağacı (Trie - Önek Ağacı) kurar. 's' harfine basınca 's' ile başlayan tüm dalları, 'i' harfine basınca 'si' ile başlayan dalları görür. Üstelik her dalın tepesinde en çok aranan ilk 5 kelime altın tepside hazır tutulur. Siz tuşa bastığınız anda 2 milisaniyede liste önünüze dökülür!

🇬🇧 ENGLISH (ELI5)

The instant you type 'sys' into Google, a dropdown flashes: 'system design', 'system requirements'. With a corpus of billions of search terms, Google cannot run a database SQL scan on every keystroke! Instead, it navigates an in-memory Prefix Tree (Trie). Typing 's' lands on the 's' branch; typing 'y' traverses to 'sy'. Crucially, every tree node pre-caches the Top-5 most popular global search phrases right on its branch. The moment your finger hits the key, suggestions render in 2 milliseconds!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Typeahead Mimarisi: 1) Trie Veri Yapısı: Kökten yapraklara karakter bazlı dallanan ağaçtır. Arama süresi kelime uzunluğu k kadardır ($O(k)$); kelime sayısından (N) tamamen bağımsızdır. 2) Top-K Önbellekleme: Her Trie düğümünde en sık aranan ilk 5-10 kelime ve frekansı saklanır. Kullanıcı 'sis' yazdığında ağacın derinliklerine inmeden o düğümdeki hazır listeyi döner ($O(1)$). 3) Çevrimdışı Analiz Hattı (Offline Aggregation Pipeline): Trie çalışma anında güncellenmez (yavaşlatmamak için). Arama logları Kafka -> Flink/Spark ile toplanır; gece batch işinde yeni frekanslar hesaplanır ve güncel Trie bellek kümesine (Redis veya Custom C++ / Java servisi) yüklenir. 4) İstemci Tarafı Optimizasyon: Tarayıcı tarafında 200ms `debounce` (kullanıcı yazmayı bırakana kadar bekleme) ve yerel önbellek uygulanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Search Typeahead Architecture: 1) The Trie Data Structure: Character-level prefix tree providing $O(L)$ traversal time (L = query length), completely independent of total vocabulary size N. 2) Node-Level Top-K Caching: To eliminate subtree traversals during runtime lookups, each Trie node caches the Top-5 most frequent completions directly on its node struct. Querying prefix 'sys' reads the precomputed Top-5 array in $O(1)$ time. 3) Offline Aggregation Pipeline: Live search keystrokes stream into Kafka and map-reduce via Apache Flink to aggregate weekly frequency scores. Updated Trie snapshots rebuild offline and hot-swap into distributed memory clusters. 4) Client-Side Optimizations: Frontends implement a 200ms debounce buffer and local LRU cache to slash upstream keystroke queries by 60%.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Top-K Destekli Trie (Önek Ağacı) Implementasyonu
import java.util.*;

public class TypeaheadTrie {
    private static class TrieNode {
        Map<Character, TrieNode> children = new HashMap<>();
        List<String> topSuggestions = new ArrayList<>(); // Önceden hesaplanmış Top-K
    }

    private final TrieNode root = new TrieNode();

    public void insert(String phrase) {
        TrieNode curr = root;
        for (char c : phrase.toLowerCase().toCharArray()) {
            curr = curr.children.computeIfAbsent(c, k -> new TrieNode());
            if (!curr.topSuggestions.contains(phrase)) {
                if (curr.topSuggestions.size() < 3) {
                    curr.topSuggestions.add(phrase);
                }
            }
        }
    }

    public List<String> getSuggestions(String prefix) {
        TrieNode curr = root;
        for (char c : prefix.toLowerCase().toCharArray()) {
            curr = curr.children.get(c);
            if (curr == null) return List.of();
        }
        return curr.topSuggestions;
    }
}

void main() {
    var trie = new TypeaheadTrie();
    trie.insert("sistem tasarimi");
    trie.insert("siber guvenlik");
    trie.insert("sistem gereksinimleri");
    trie.insert("spring boot action");

    IO.println("Öneri ('sis'): " + trie.getSuggestions("sis"));
    IO.println("Öneri ('sp'):  " + trie.getSuggestions("sp"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta Trie ağacını her kullanıcı arama yaptıkça canlı (real-time) olarak güncelleyeceğinizi söylemeyin! Milyonlarca eşzamanlı yazma işlemi Trie ağacında kilitlenme (Lock Contention) yaratır ve belleği tüketir. Güncellemeler daima çevrimdışı batch pipeline ile yapılmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose updating Trie frequencies synchronously on live search queries. Concurrent lock contention on root nodes will crater latency. All frequency score recalculations must occur asynchronously via offline batch pipelines.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Trie (Önek Ağacı) arama süresini kelime sayısından bağımsız olarak O(kelime_uzunluğu) yapar.

• Düğüm üzerinde Top-K önbellekleme arama anında tüm ağacı tarama zorunluluğunu kaldırır.

• İstemci tarafında Debouncing (200ms) uygulamak sunucuya gelen istek yükünü %60 azaltır.

🎯 Key Takeaways for the Interview

• Tries bound search latency to O(prefix_length), completely independent of dictionary scale.

• Pre-computing Top-K suggestions at each Trie node delivers instant O(1) query returns.

• Client-side debouncing buffers rapid typing, cutting keystroke query volume by 60%+.

48

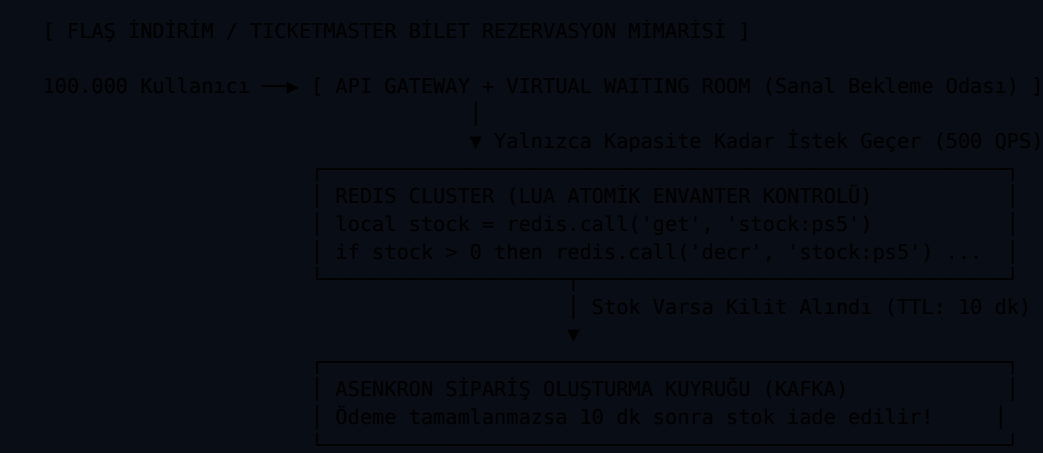
BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

E-Ticaret Flaş İndirim ve Bilet Satış Sistemi (Flash Sale / Ticketmaster)

Designing a Flash Sale & Ticketing System (Ticketmaster / Amazon Prime Day)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Stadyumda bir rock konseri için sadece 1.000 bilet olduğunu ve kapıda 100.000 kişinin beklediğini düşünün. Kapıyı ardına kadar açarsanız 100.000 kişi içeri dalar, gişeyi kırar ve izdihamda herkes ezilir (Veritabanı çöküşü!). Çözüm: 1) Kapının önüne döner bir bariyer (Virtual Waiting Room) koyarsınız. İçeriye sadece sırayla 1.000 kişi alınır. 2) Gişedeki biletçi sepete eklenen bilete 10 dakikalık bir geri sayım sayacı yapıştırır (Geçici Kilit). 10 dakika içinde parayı ödemezseniz bilet elinizden alınır ve sıradaki kişiye verilir!

🇬🇧 ENGLISH (ELI5)

Picture a stadium rock concert with only 1,000 tickets, but a mob of 100,000 fans waiting outside. If you throw open the gates, 100,000 people trample the box office and smash the counters (Database meltdown!). The solution: 1) Set up a velvet rope turnstile outside (Virtual Waiting Room). Fans queue in order; only batches of 500 enter at a time. 2) When a fan grabs a seat, the system slaps a 10-minute countdown timer on the ticket (Atomic Reservation with TTL). If they don't enter credit card info within 10 minutes, the hold expires and the seat returns to the pool!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Flaş İndirim Mimarisi: 1) Sanal Bekleme Odası (Virtual Waiting Room): Cloudflare Waiting Room veya Kafka kuyrukları ile kullanıcılar adil bir sıra numarası (FIFO) ile bekletilir; veritabanı kapasitesi kadar kullanıcı içeri alınır. 2) Bellek İçi Atomik Envanter (In-Memory Atomic Inventory): Veritabanına `UPDATE products SET stock = stock - 1 WHERE id = 1 AND stock > 0` yapmak satır kilidi (Row Lock) yüzünden DB'yi kilitler. Stok Redis'te tutulur ve Lua Script ile atomik olarak düşürülür (`DECR`). Sıfırın altına düşmesine asla izin verilmez. 3) Geçici Rezervasyon ve Dağıtık TTL: Kullanıcı bilet seçtiğinde bilet 10 dakika rezerve edilir (`SET seat:42:lock user_101 EX 600`). Ödeme başarılı olursa DB'ye kesin satış yazılır. Ödeme başarısız olursa veya 10 dakika dolarsa TTL biter ve bilet havuza geri döner.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Flash Sale & Ticketing Architecture: 1) Virtual Waiting Room (Fair Queuing): Deploys an edge ingress queue (Cloudflare Waiting Room) metering traffic based on backend throughput capacity, preventing database saturation. 2) Atomic In-Memory Stock Depletion: Executing relational database updates (`UPDATE stock SET qty = qty - 1 WHERE qty > 0`) creates severe row-level lock contention. Inventory is decremented in-memory inside Redis via atomic Lua scripts in microsecond speeds. 3) Two-Phase Inventory Hold (Reservation TTL): Stock is not permanently sold until payment commits. Grabbing an item creates a temporary hold with a 10-minute expiration (`SET item:hold:user EX 600`). If payment confirmation completes, a Kafka event finalizes the database commit; if the timer expires, the hold releases atomically back to available inventory.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Flaş İndirim Atomik Stok ve Geçici Rezervasyon Motoru
import java.util.concurrent.*;

public class FlashSaleEngine {
    private int availableStock;
    private final ConcurrentMap<String, Long> activeReservations = new ConcurrentHashMap<>();

    public FlashSaleEngine(int initialStock) {
        this.availableStock = initialStock;
    }

    public synchronized boolean tryReserveItem(String userId, long holdDurationMillis) {
        if (availableStock <= 0) {
            IO.println("❌ STOK TÜKENDİ! Kullanıcı: " + userId);
            return false; // Stok bitti
        }

        availableStock--;
        activeReservations.put(userId, System.currentTimeMillis() + holdDurationMillis);
        IO.println("✔ Stok Rezerve Edildi: " + userId + " (Kalan Stok: " + availableStock + ")");
        return true;
    }

    public synchronized void cancelExpiredReservation(String userId) {
        Long expiration = activeReservations.remove(userId);
        if (expiration != null) {
            availableStock++; // Stok havuza iade edildi
            IO.println("⏱ Süre Doldu! Stok İade Edildi: " + userId + " (Güncel Stok: " +
availableStock + ")");
        }
    }
}

void main() {
    var sale = new FlashSaleEngine(2); // Toplam 2 adet PS5
    sale.tryReserveItem("user_ali", 600_000);
    sale.tryReserveItem("user_veli", 600_000);
    sale.tryReserveItem("user_can", 600_000); // Stok kalmadı!

    sale.cancelExpiredReservation("user_ali"); // Ali ödemedi -> Stok iade!
    sale.tryReserveItem("user_can", 600_000);  // Can artık alabildi!
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Stoku doğrudan MySQL'de kilit açarak (SELECT FOR UPDATE) yönetirim' demeyin! Saniyede 100.000 kişinin aynı satırı kilitlemeye çalışması veritabanında Connection Pool tükenmesine ve sistemin çökmesine neden olur. Stok kontrolü daima Redis'te atomik yapılmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch relational `SELECT ... FOR UPDATE` row locks for flash sales. 50,000 concurrent threads vying for the exact same database row lock will crash the connection pool in seconds. Manage hot stock counts in Redis memory via atomic Lua scripts.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Sanal Bekleme Odası (Virtual Waiting Room) veritabanını aşırı yükten korur.

• Stok düşürme işlemi veritabanı yerine Redis üzerinde atomik Lua scripti ile yapılır.

• Geçici rezervasyon ve TTL sayesinde ödeme yapmayan kullanıcıların ürünleri havuza döner.

🎯 Key Takeaways for the Interview

• Virtual waiting rooms throttle user concurrency to match backend database processing limits.

• Atomic in-memory decrements in Redis eliminate relational database row-locking contention.

• Reservation TTL windows release unclaimed inventory automatically back to waiting fans.